Theory of Algorithms – Introduction

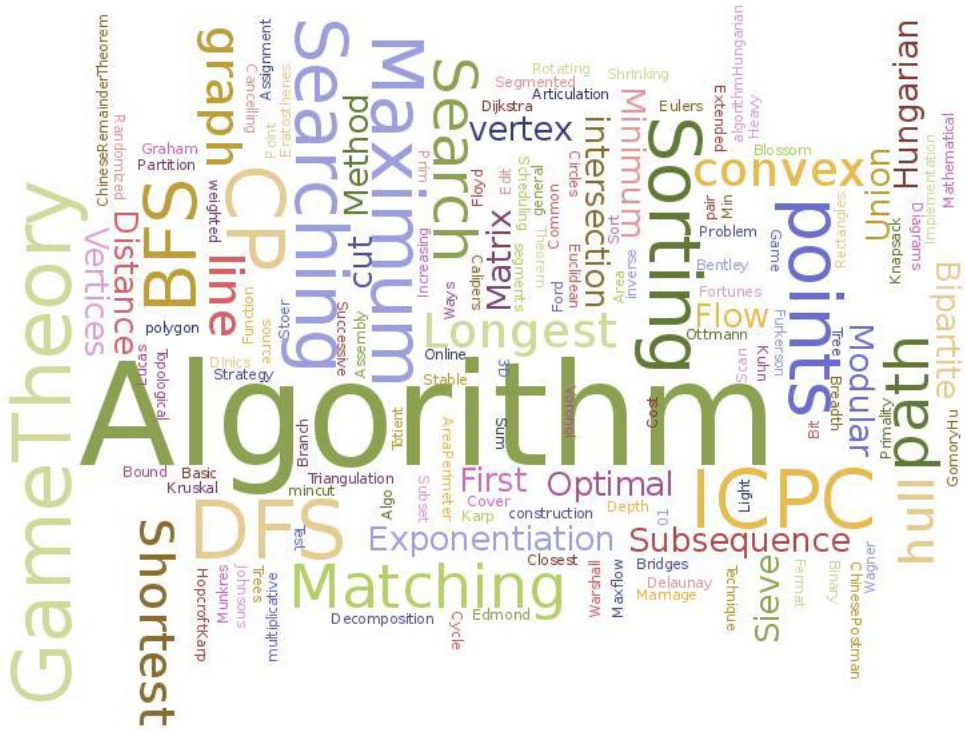
Patrick Marais
Department of Computer Science
University of Cape Town
patrick@cs.uct.ac.za





What is an Algorithm?

- Sequence of unambiguous instructions for solving a well-defined problem
  - Algorithms are guaranteed to terminate if the input is valid
- Can we write an algorithm that given any program as input, will tell us whether or not, for any input, that program stops?



Algorithm Features

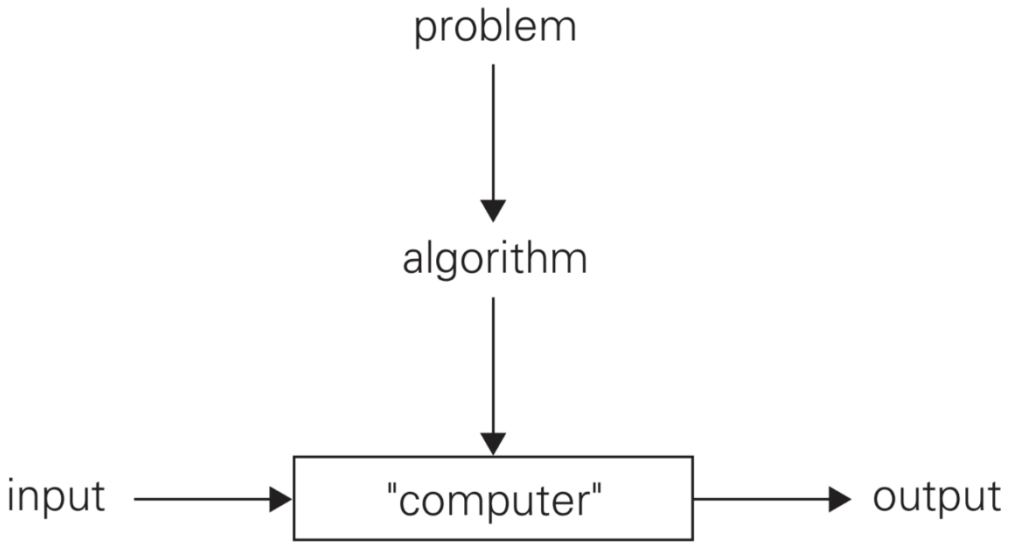
- Finite
  - Terminates after a finite number of steps
- Definite
  - Rigorously and unambiguously specified
- Input
  - Valid inputs are clearly specified
- Output
  - Can be proved to produce the correct output given a valid input
- Effective
  - Steps are sufficiently simple and basic (comprehensibility)

Notion of an Algorithm

- Each step of the algorithm must be unambiguous
- The range of inputs must be specified carefully
- The same algorithm can be represented in different ways

AND

- Several algorithms for solving the same problem may exist - with different properties

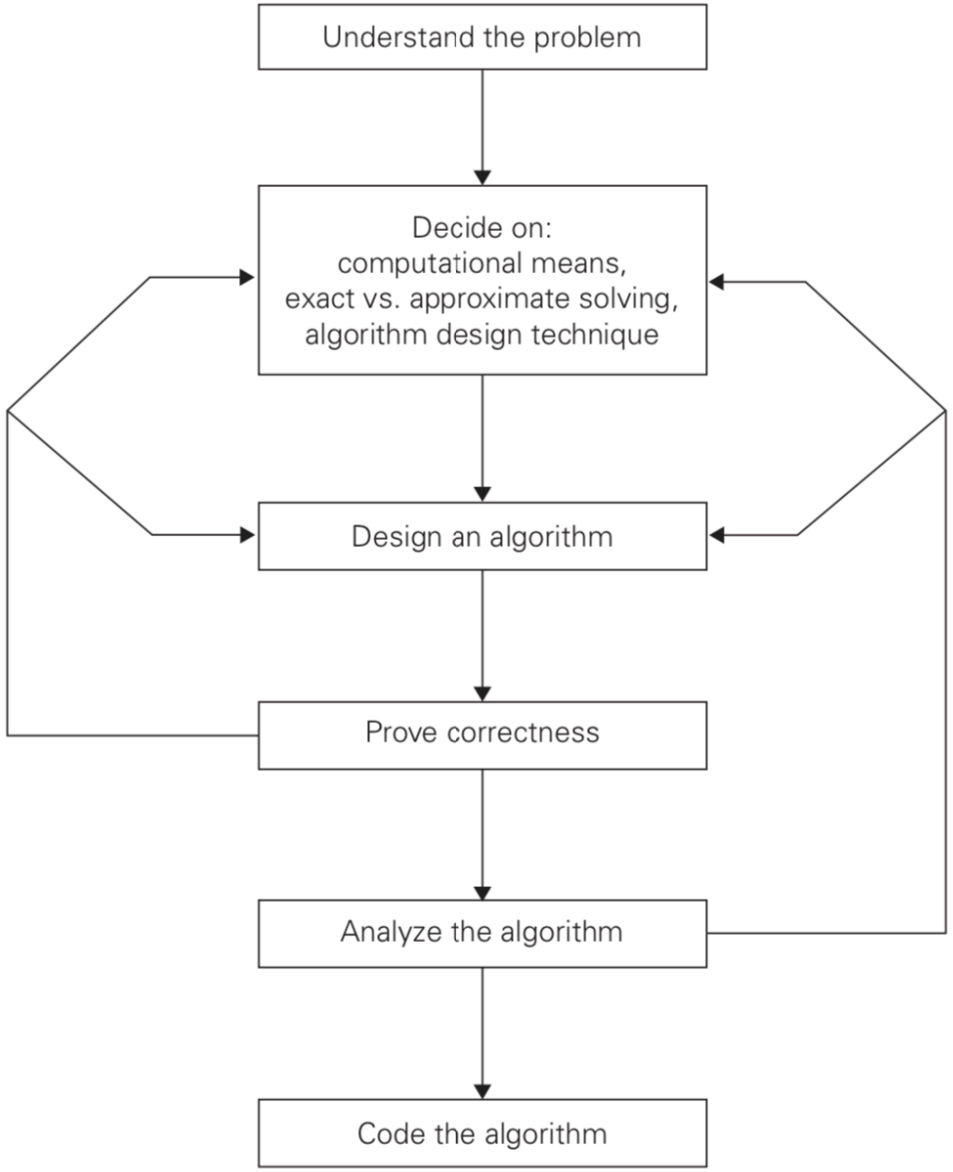


Better Algorithms

BUT

No point in finding fastest algorithm if:

- That part of the system is not the bottleneck
- The program will only be run very few times (e.g. reconciling & merging databases when 2 companies merge)
- Time is not an issue (e.g. can run overnight)

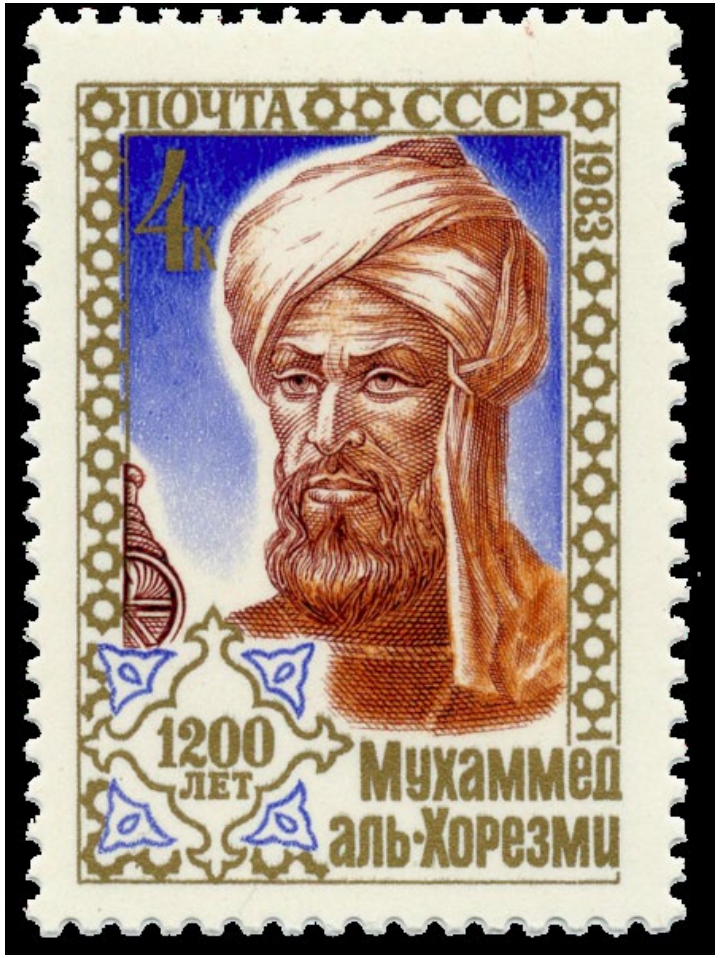


Algorithm Design Basics

- Specify the algorithm
  - Pseudocode typically
- Data structures are important too
- Design principles
  - Simplicity
  - Generality – range of inputs, special cases
  - Optimality – no other algorithm can do better
- Formal Algorithm Analysis:
  - Efficiency – time and space

History

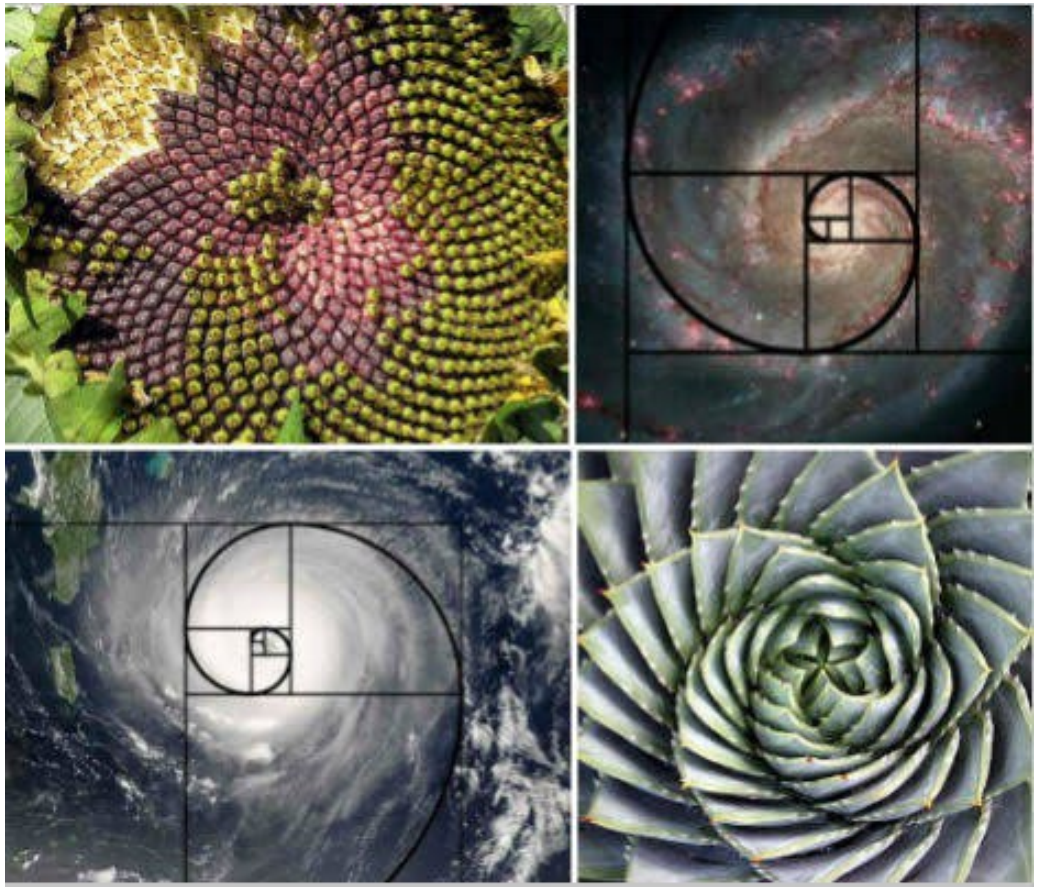
- 1600 BC
  - Babylonians developed algorithms for factorization and square roots
- 300 BC
  - Euclid developed several algorithms
- Muḥammad ibn Mūsā al-Khwārizmī (780-850 AD)
  - Contributions to algebra, decimal numbers and much else
  - "Algorithm" derived from Latin form of his name



Fibonacci

- The Italian, Leonard Fibonacci, was responsible for introducing the decimal system to the west in the 15th century
- He also defined this series
  - 0, 1, 1, 2, 3, 5, 8,13, 21, 34, 55, etc.

$$F(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$



Typical Problem Domains

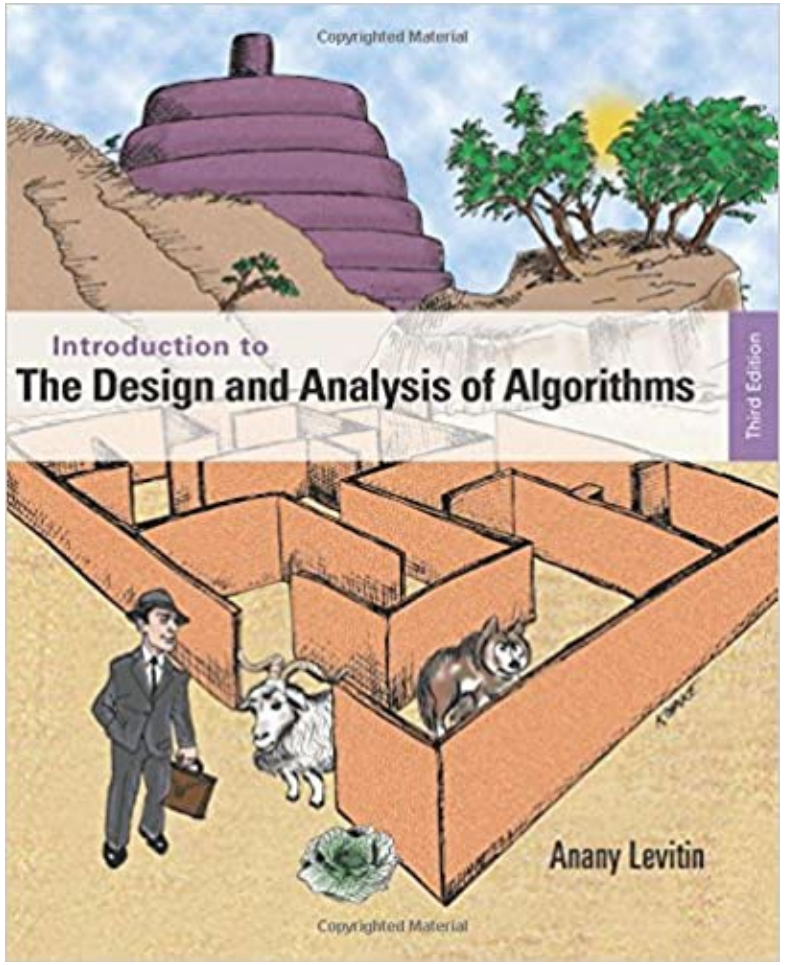
- Sorting and searching
- String Processing
- Graph Problems
- Combinatorial Problems
  - E.g. List all the permutations of a set
- Geometric Problems
  - Closest-Pair, Convex-Hull
  - GPS: Least number of roads/shortest path to get from A to B?
- Numerical Problems
  - Solving systems of equations, random numbers, matrix multiplication

Types of Algorithms

- Brute force
  - Follow definition, try all possibilities
- Decrease & conquer
  - Solve larger instance in terms of smaller instance
- Divide & conquer
  - Break problem into distinct subproblems
- Transformation
  - Convert problem to another one
- Space/Time Trade-off
  - Use additional data structures
- Dynamic programming
  - Break problem into overlapping subproblems
- Greedy
  - Repeatedly do what is best now

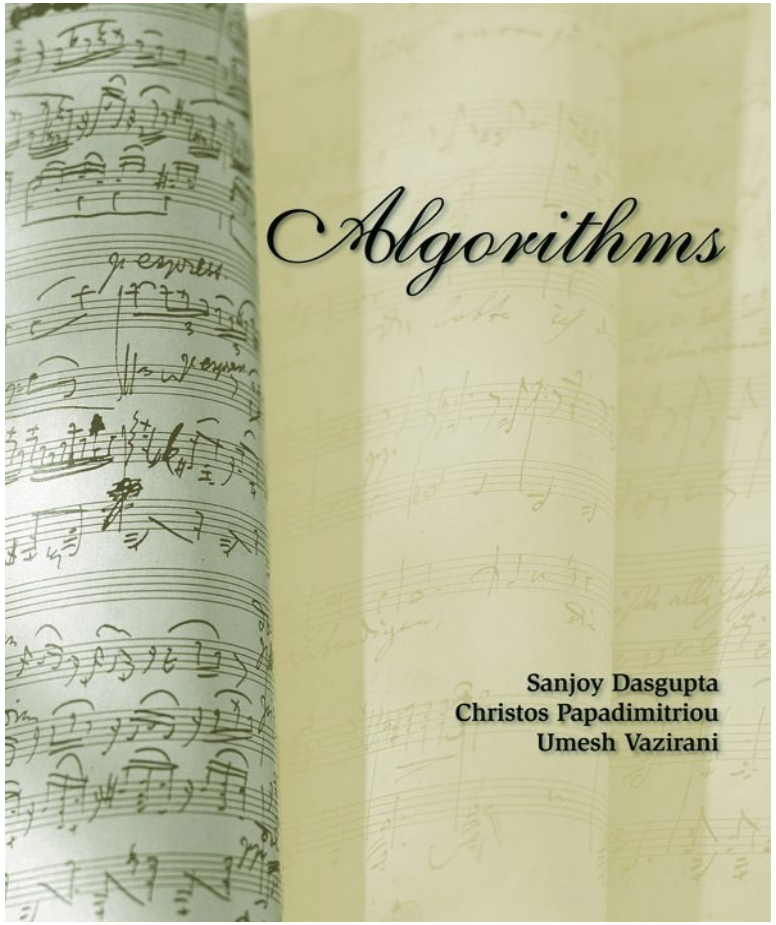
Prescribed Textbook

- Levitin, "Introduction to the Design and Analysis of Algorithms" [3rd edition]



Additional Resources

- Dasgupta et al., "Algorithms"
  - Freely available at http://algorithmics.lsi.upc.edu/docs/Dasgupta-Papadimitriou-Vazirani.pdf
  - Basis of Berkely Algorithms course
- Knuth, "The Art of Computer Programming"
  - Bible of Algorithms, in four volumes (and counting)
  - Expensive



Learning for the Test

- If you learn and take an interest, you'll do well
- Go through the slides
  - If you don't understand a slide, refer to the Levitin book
  - Occasionally slides will refer directly to a section in the Levitin book and give no further explanation. In which case read and understand that section
  - If slides reference another source, e.g. Dasgupta, you shouldn't have to go to it, unless you can't understand the slide
- NB: Slides contains things not in Levitin. Examinable unless explicitly stated otherwise
- Ask questions in class or on forums

Closed Practicals

- They're hard. It's an opportunity to improve your programming
- Don't stress out. Practice. Do your best
- Week one:
  - Brute force – Easiest, but not trivial
- Week two:
  - Divide & Conquer/recursion – Hard
- Week three:
  - Dynamic programming – Very hard, but….
- It must be your own work

Theory of Algorithms: Examples



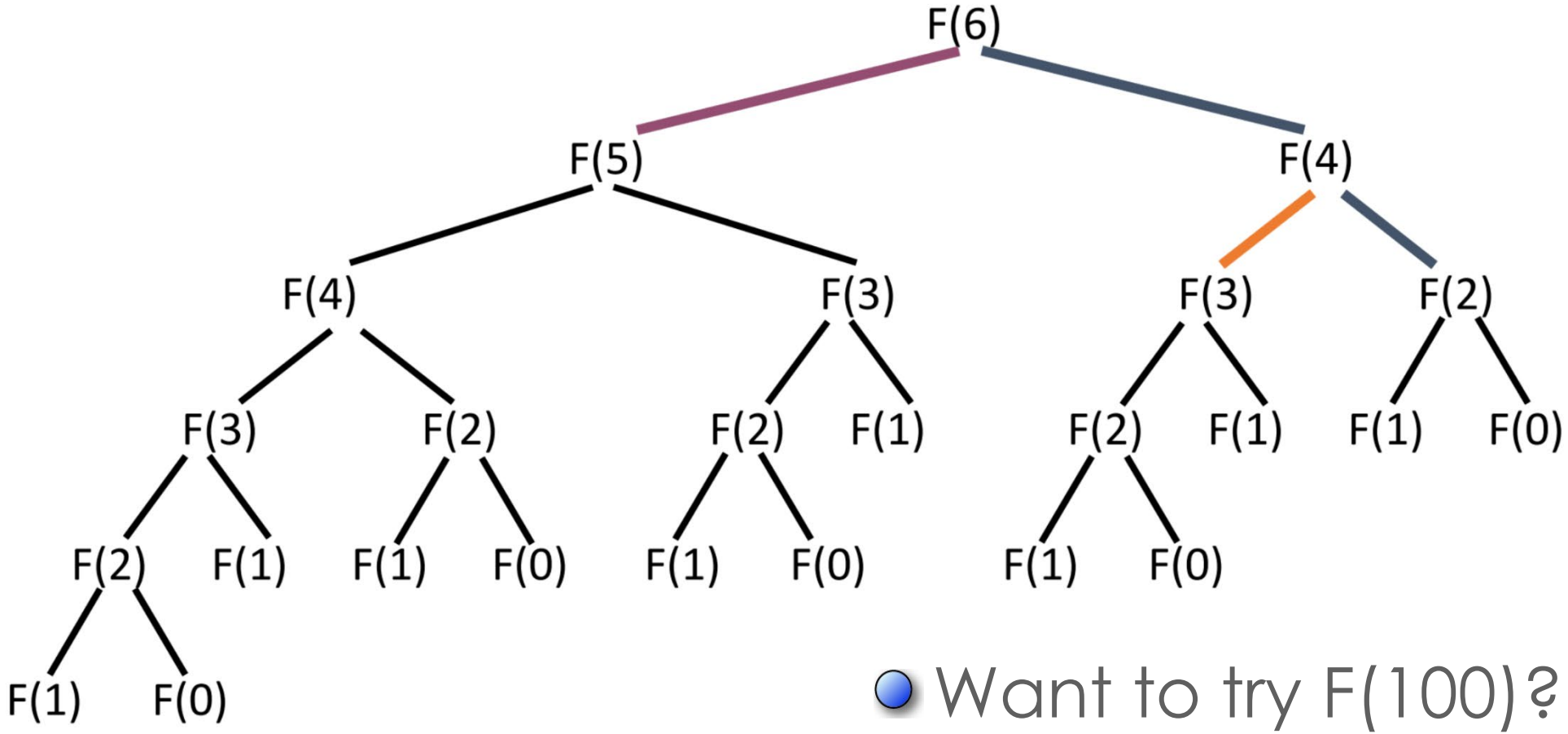


Brute Force Solution

```
public long fibo(long n) {
     if (n == 0) return 0;
     if (n == 1) return 1;
     return fibo(n-1) + fibo(n-2);
}
```

- Seems okay

Fibonacci F(6) using Brute Force



- Want to try F(100)?

Better Solution v1

```
Fibo(n):
     // Computes nth Fibonacci number
     // Input: Non-negative integer n
     // Output: nth Fibonacci number

     F[0] = 0
     F[1] = 1
     for i = 2 to n do:
          F[i] = F[i - 1] + F[i - 2]
     return F[n]
```

Better Solution v2

```
public BigInteger fibo(int n) {
      if (n==0) return 0; if (n==1) return 1;
      BigInteger [] table = new BigInteger[3];
      table[0] = new BigInteger("0");
      table[1] = new BigInteger("1");

      for (int i=2; i<n; i++) {
            table[2] = table[1].add(table[0]);
            table[0] = table[1];
            table[1] = table[2];
      }
      return table[2];
}
```

- In what ways is this better or worse than v1?

By Formula

```
public long fibo(int n) {
      return Math.round(
            (1.0/Math.sqrt(5.0))*
            Math.pow(1.6180339887,n));
}
```

- Superficially, looks extremely efficient. But it has practical implementation problems
- Can you work out, or guess, the problem?
- And why *superficially* efficient?
- Incidentally 1.618 approximates the Golden ratio: (1+sqrt(5))/2

Problems with the Formula

- Using Java It calculates Fibo up to 42 correctly
- But for Fibo(43) it gives 433494436. The correct answer is 433494437
- It gets increasingly worse for higher n
- Floating point rounding problems!
- While the Fibonacci series is hardly something you're likely to use in your day-to-day work, the three solutions to it are very instructive
- We'll revisit this soon when we analyse complexity

Greatest Common Divisor

- Find two methods for solving this problem
- Given two positive integers m and n, find their greatest common denominator
  - the largest integer which divides them both without remainder
  - E.g., GCD(60, 24) = 12
  - Divisors(60) = [1, 2, 3, 4, 5, 6, 10, 12, 15, 20, 30, 60]
  - Divisors(24) = [1, 2, 3, 4, 6, 8, 12, 24]

Euclid's Solution

```
int gcd(int m, int n) {
    if (n==0) return m;
    return gcd(n, m % n);
}
```

- Only 2300 years old

```
gcd(14142, 3131)
= gcd(3131, 1618)
= gcd(1618, 1513)
= gcd(1513, 105)
= gcd(105, 43)
= gcd(43, 19)
= gcd(19, 5)
= gcd(5, 4)
= gcd(4, 1)
= gcd(1, 0)
= 1
```

Euclid's Solution without Recursion

```
int gcd(int m, int n) {
      int r;
      while (n != 0) {
            r = m % n;
            m = n;
            n = r;
      }
      return m;
}
```

- No more efficient than the recursive solution for some compilers, because the recursive solution is tail recursive
  - A C++, compiler will optimise it and remove the recursion

GCD v2

```
int gcd(int m, int n) {
     int t  = min(m,n);
     while(m % t > 0 || n % t > 0) {
          t--;
     }
     return t;
}
```

- GCD(60, 42) tries 42, 41, ...